Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Segmentation Strategies for Information Retrieval

No Author Given

No Institute Given

Abstract. Document chunking is a critical preprocessing step in dense retrieval systems, yet the design space of chunking strategies remains poorly understood. Recent work has proposed novel approaches ranging from LLM-guided chunking methods such as DenseX [**?**] and Lumber-Chunker [**?**], to contextualized chunking strategies like Late Chunking [**?**], which generates embeddings before segmentation to preserve contextual information. However, existing methods have been evaluated in isolation across different benchmarks, making direct comparisons difficult.
This paper presents a systematic framework that unifies existing chunking strategies along two key dimensions: (1) the *order* of chunking and embedding operations (pre-embedding vs. contextualized chunking), and (2) the *segmentation methods* (traditional fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided approaches). We reproduce and evaluate representative methods from each category across two distinct retrieval paradigms: single-document needle-in-a-haystack retrieval and multi-document web-scale retrieval. Our comprehensive evaluation reveals that ... Our code and evaluation benchmarks are publicly available at `https://xx`.

Keywords: Chunking · Document Ranking · Re-ranking.

1 Introduction

Dense retrieval systems, which represent queries and documents as low-dimensional vectors, have become a core part of modern information retrieval. They power a wide range of applications, from web search engines to advanced retrieval-augmented generation systems. A key challenge in using these systems is fitting long documents into the fixed-size input windows of embedding models. This requires **document chunking**: the process of dividing documents into smaller, meaningful units for indexing and retrieval. The choice of a chunking strategy is not just a preprocessing step; it is an important decision that greatly affects how well retrieval works. Chunks that are too small may not have enough context to answer a query, while oversized chunks can make it harder to find the relevant information and may be too large for the model to process.

Chunking has developed in two main ways: the method of segmentation and the approach to embedding. Traditionally, systems use a *pre-embedding chunking* approach, where documents are first divided and each resulting chunk is then encoded on its own. Within this approach, segmentation methods range from simple fixed-size windows to more advanced techniques based on linguistic boundaries (e.g., sentences or paragraphs [**?**]), changes in meaning [**?**], and even guidance from large language models (LLMs) [**?**,**?**]. More recently, the introduction of embedding models with longer context windows that support thousands of tokens [**?**,**?**,**?**] has allowed for a new *contextualized chunking* approach. Methods like Late Chunking [**?**] are a good example of this change: they first encode a whole document to create context-aware embeddings and only then divide it, keeping useful information that connects the chunks.

Despite this rapid development, progress has been disorganized because there has been no standard way to evaluate these methods. New chunking strategies are often tested in different and unconnected ways, making it impossible to compare them directly. For example, proposition-based methods were evaluated on question-answering datasets using models with a 512-token limit [**?**]. In contrast, LLM-guided methods like LumberChunker were benchmarked on a single-book retrieval task using private embedding APIs [**?**], while Late Chunking was tested on standard BEIR passage retrieval benchmarks [**?**]. These differences cover embedding models, datasets, and even the type of retrieval task (e.g., searching through many documents vs. a single document). As a result, it is unclear whether the differences in performance come from the actual strengths of the chunking strategies themselves or from these other factors in the experiments.

To address these gaps and provide a clear, evidence-based understanding of document chunking, we conduct a careful study guided by the following research questions:

RQ1 *How do different segmentation methods compare in standard multi-document retrieval?* We evaluate fixed-size, sentence/paragraph-based, semantic boundary-based, and LLM-guided chunking methods using traditional pre-embedding chunking on standard BEIR benchmarks.

RQ2 *Do segmentation methods generalize across retrieval paradigms?* We apply the same segmentation methods to single-document needle-in-a-haystack retrieval, testing whether the relative strengths observed in multi-document settings hold when the retrieval scope is constrained to individual documents.

RQ3 *Can contextualized chunking provide consistent improvements across different configurations?* We apply contextualized chunking to all segmentation methods from RQ1 across both retrieval paradigms from RQ2 to determine whether contextualized chunking provides universal benefits.

RQ4 *What factors determine when contextualized chunking outperforms pre-embedding approaches?* We implement Late Chunking as a representative contextualized method and systematically vary chunk size, overlap ratios, document types, and query characteristics to measure both retrieval effectiveness and computational overhead.

Through investigating these research questions, we (1) establish comprehensive baselines for segmentation methods in multi-document retrieval (**RQ1**), (2) identify which chunking strategies are robust across different retrieval paradigms (**RQ2**), (3) assess whether contextualized chunking provides universal improvements (**RQ3**), (4) uncover the key factors that determine when contextualized chunking justifies its computational overhead (**RQ4**), and (5) release our complete implementation and evaluation framework to facilitate reproducibility and future research.

2 Benchmarking Document Chunking Strategies

Document chunking translates raw text into retrievable units and therefore sits at the critical interface between corpus preparation and dense retrieval. To reason about the wide range of published chunkers, we organize the design space along two orthogonal dimensions: (1) **chunk boundary granularity**, i.e., how textual boundaries are detected, and (2) **embedding order**, i.e., whether chunking happens before or after contextual representations are produced. The taxonomy below provides common terminology for our experimental comparisons and highlights where new techniques diverge from conventional pipelines.

2.1 Taxonomy of Chunking Strategies

We divide chunking strategies into structure-preserving heuristics and semantically informed approaches. Both families can be instantiated with different hyper-parameters (window size, overlap, confidence thresholds), but they reflect distinct assumptions about how meaningful retrieval units should be constructed.

Structure-Preserving Heuristics Heuristic chunkers rely on observable surface structure and have historically dominated production retrieval systems because they are deterministic, fast, and easy to parallelize **??**. We consider three representatives:

- **Fixed-size windows** partition the document into contiguous spans of a preset token length, optionally with overlap to soften boundary effects. Their main appeal is hardware efficiency because index sharding and batch encoding are straightforward, but they may cut through coherent discourse units, creating embeddings that blend unrelated topics [**?**].
- **Paragraph boundaries** treat author-provided paragraph breaks as chunk anchors. This respects document-level structure with negligible preprocessing cost and often aligns with topical shifts in well-edited corpora such as news or scientific articles **??**.
- **Sentence blocks** aggregate a small, fixed number of consecutive sentences. This yields finer-grained segments that are attractive for QA-style tasks where precise answer localization matters, though the shorter context can harm recall when evidence is distributed across a broader span **??**.

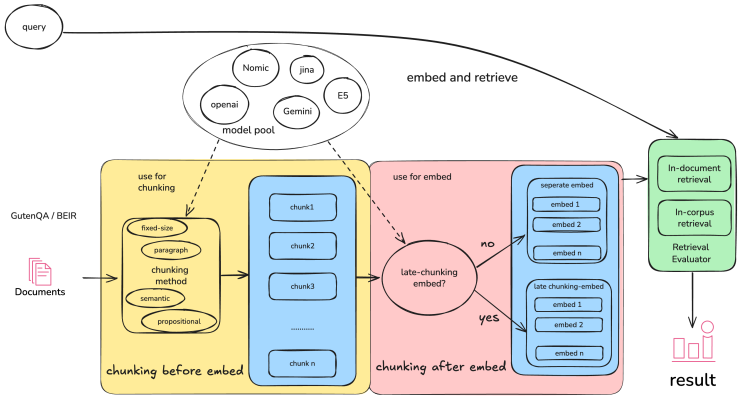

Fig. 1: Unified framework for evaluating document chunking strategies across retrieval paradigms. Documents from BEIR and GutenQA datasets are segmented using one of several methods: fixed-size, paragraph-based, sentence-based, semantic sentence-based, propositional, or LLM-guided chunking (e.g., LumberChunker). Depending on the configuration, embeddings are generated either before chunking (*pre-embedding chunking*) or after full-document encoding (*contextualized chunking*, e.g., Late Chunking). Resulting chunk embeddings are indexed and evaluated under two retrieval settings: (1) *in-document retrieval* (needle-in-a-haystack) and (2) *in-corpus retrieval* (multi-document). The pipeline supports multiple embedding and generative model families (e.g., E5, Jina, Nomic, OpenAI) [**?**], enabling controlled comparison across chunking paradigms and model capacities.

These heuristics illustrate the trade-off between computational efficiency and semantic completeness. Because they ignore discourse cohesion beyond surface cues, effectiveness can degrade when relevant information straddles boundary decisions, motivating more adaptive chunkers.

Semantically Informed and LLM-Guided Strategies Recent work leverages contextual representations or large language models (LLMs) to place boundaries at semantic inflection points instead of predetermined offsets. The intuition is to preserve the internal coherence of each chunk so that dense retrievers operate on self-contained units **??**.

- **Semantic sentence chunking** computes similarity between adjacent sentences using lightweight embedding models and inserts a boundary when topical drift exceeds a threshold [**?**]. The resulting variable-length segments adapt to discourse structure while remaining automation-friendly.
- **Proposition-level chunking** decomposes text into minimal factual statements ("propositions") via prompt-engineered LLMs [**?**]. Each chunk aims to capture an atomic claim, which is beneficial for factoid retrieval but can explode the number of segments and increase indexing cost.

– **LLM-guided chunking (e.g., LumberChunker)** explicitly asks an LLM to mark coherent sections after reading the surrounding context [**?**]. This approach can exploit discourse cues that embeddings miss (narrative arcs, rhetorical structure), yet its cost scales with token limits and inference latency, constraining applicability to large corpora **??**.

These strategies sacrifice some determinism for semantic fidelity. In our experiments we instantiate each chunker using publicly released implementations wherever possible to ensure comparability with prior reports **??**.

2.2 Pre-Embedding vs. Contextualized Chunking

The second axis of our taxonomy concerns **when** chunking occurs relative to representation learning. Traditional *pre-embedding* pipelines segment first, then encode each chunk independently **??**. This design enables embarrassingly parallel encoding and straightforward inverted-file style indexing, but it strips away inter-chunk context because the encoder never observes more than one span at a time. When relevant evidence spans multiple segments, the resulting embeddings may be incomplete or inconsistent.

Contextualized chunking (also called late chunking) inverts this order. Long-context encoders process the entire document (or large overlapping windows) to produce token-level embeddings that already incorporate global information [**?**]. Afterward, chunk boundaries, whether fixed windows or LLM-derived spans, are applied to the contextualized tokens, and segment embeddings are aggregated through pooling or attention-based summarization **??**. This preserves cross-span cues (e.g., entity co-reference, causal chains) while still exposing chunk-level vectors to downstream retrievers.

The choice between these paradigms affects computational budgets and memory footprints. Pre-embedding scales gracefully to web-scale corpora but can underperform when the optimal chunk size exceeds an encoder's maximum context length. Contextualized chunking mitigates that issue at the cost of longer forward passes, additional pooling logic, and more complex engineering around sliding windows for very long documents **??**. Our evaluation therefore contrasts both orders under matched chunking policies to isolate when the additional expense is justified.

2.3 Mismatch in Evaluation Protocols

Prior work on chunking spans heterogeneous evaluation settings, complicating direct comparison of reported gains [**?**,**?**,**?**]. Two paradigms dominate:

– **In-document (needle-in-a-haystack) retrieval** restricts the search space to a single long document and measures how quickly a method surfaces the relevant passage. Datasets such as GutenQA stress precise localization, rewarding chunkers that preserve narrative continuity and suppress noisy overlap **??**.

- **In-corpus (multi-document) retrieval** evaluates ranking quality across a large, heterogeneous collection like MS MARCO or the BEIR suite [**?**,**?**]. Here, scalability, indexing efficiency, and robustness to domain shifts become critical, sometimes favoring simpler chunk boundaries that align with batching constraints.

The literature often varies additional factors simultaneously, including embedding models, context-window limits, and negative sampling strategies, leaving open whether the observed improvements stem from the chunker itself or from confounding experimental conditions **??**. By reproducing representative methods under a single unified framework, we disentangle these influences and quantify which chunking decisions truly matter for each retrieval regime.

3 Experimental Setup

We evaluate every chunking strategy within the unified pipeline introduced in Figure 1. This section first details the configuration of that pipeline and then summarizes the datasets and evaluation methodology used to assess each variant.

3.1 Pipeline Configuration

Chunking Modules Each chunking strategy from Section 2 is instantiated with a single implementation that applies to all datasets.

- **Fixed-size windows:** Documents are divided into 512-token spans with no overlap. Confirm whether overlap or stride tuning is explored.
- **Paragraph boundaries:** Segments follow newline-delimited paragraphs after lightweight normalization that collapses consecutive newlines.
- **Sentence blocks:** We group five consecutive sentences produced by spaCy's English sentence segmenter. Verify tokenizer language model version.
- **Semantic sentence chunking:** We reuse the open-source implementation described in **??**, using jina-embeddings-v2-small-en to score adjacency and place a boundary when cosine similarity falls below the default threshold.
- **LumberChunker:** We prompt Gemini-1.5-Flash for GutenQA and GPT-4.1-Nano for BEIR documents, mirroring settings published in [**?**]. Chunk boundaries are extracted from JSON responses and aligned back to token spans.
- **Proposition-level chunking:** For GutenQA we cascade LumberChunker with a second GPT-4.1-Nano prompt that enumerates propositions inside each coarse segment; for BEIR we invoke the proposition prompt directly. Document prompt templates and temperature.

Every method emits a sequence of span offsets that is consumed by the encoder modules described next.

Encoder Settings We evaluate four encoder families (jina-embeddings-v2-small-en, jina-embeddings-v3, nomic-embed-text-v1, and multilingual-e5-base) in two modes.

- **Regular encoder (pre-embedding):** Each chunk is encoded independently, and the resulting vectors are normalized and indexed.
- **Late encoder (contextualized):** The full document, or overlapping slices when necessary, is passed through the encoder to obtain token-level embeddings. Span offsets from the chunker are applied afterward, and chunk vectors are produced via mean pooling. Assess whether attention pooling improves stability.

When encoders require instruction prefixes (for example, the E5 instruction token), we prepend the prefix only to the first span and append suffix tokens to the final span so that intermediate chunks correspond to contiguous document text. All encoders operate with mixed precision (FP16) on GPU unless otherwise noted. Record hardware specifications.

Retrieval Pipeline Chunk vectors are stored in a FAISS flat index with cosine similarity. Retrieval consists of top-k dense retrieval followed by optional re-ranking when the dataset protocol requires it. For BEIR tasks we retrieve $k = 100$ chunks per query and evaluate ranking directly on dense scores; for GutenQA we retrieve $k = 64$ chunks and compute metrics at the chunk level. Add ablation on larger k values.

To map chunk scores back to documents we aggregate by maximum score unless the benchmark specifies passage-level evaluation. Negative sampling and index construction are performed per encoder to avoid cross-model interference.

Handling Long Documents Encoder context windows impose practical limits on segment length. In the pre-embedding setting we ensure every chunk fits within the model limit by truncating any residual tokens conservatively. For contextualized chunking we adopt the sliding-window approach from [**?**]: long documents are split into overlapping spans that fit the model window, each span is encoded independently, and chunk boundaries are applied within the span before pooling. Prefix tokens appear only in the first span and suffix tokens only in the last to maintain alignment.

3.2 Datasets and Evaluation

Datasets We consider two complementary retrieval paradigms to stress-test generalization.

- **In-document retrieval** uses GutenQA [**?**], which pairs long-form literary works with question-answer annotations. Each query is restricted to one source document, emphasizing fine-grained localization. Insert summary statistics for GutenQA (number of books, average length).

- **In-corpus retrieval** draws on a diverse subset of BEIR corpora [**?**]: FiQA, ArguAna, SciDocs, TREC-COVID, GutenQA, SciFact, and NFCorpus. These collections cover financial forums, argumentative essays, scientific abstracts, and biomedical literature. Provide per-dataset document and query counts.

All corpora are lowercased and tokenized using the default pretrained tokenizer associated with each embedding model. We preserve dataset-specific query and relevance annotations without modification.

Evaluation Metrics We report Discounted Cumulative Gain at rank 10 (DCG@10) for GutenQA, consistent with long-context retrieval benchmarks [**?**], and normalized DCG at rank 10 (nDCG@10) for BEIR datasets [**?**]. DCG@10 measures the quality of the top 10 retrieved chunks by summing graded relevance values with logarithmic discounting: $\text{DCG@10} = \sum_{i=1}^{10} \frac{2^{\text{rel}_i}-1}{\log_2(i+1)}$, where rel_i is the relevance label of the chunk at rank i. nDCG@10 divides DCG@10 by the score of an ideal ranking, yielding a value between 0 and 1 that facilitates comparison across queries. Metrics are calculated using the official BEIR evaluation toolkit with identical relevance thresholds across models. Confidence intervals are estimated via bootstrap resampling over queries. State number of bootstrap samples.

4 Result

4.1 RQ1: Impact of Chunking Strategies in Multi-Document Retrieval

To establish baseline effectiveness characteristics of different chunking strategies, we evaluated six chunkers across seven BEIR datasets using four state-of-the-art embedding models. Table 1 presents the results using traditional pre-embedding chunking, focusing on the corpus-level retrieval columns (fiqa through scidocs).

No single chunking strategy dominates across all scenarios. The effectiveness of chunkers varies significantly depending on the embedding model and dataset characteristics. Across the six corpus-level retrieval tasks, paragraph-based chunking achieves the highest average effectiveness for three out of four models (Jina-v2: 0.4303, Jina-v3: 0.4948, E5-Large: 0.4997), while fixed-size chunking performs best with Nomic embeddings (0.4828). This suggests that the optimal strategy depends on how different embedding models encode semantic boundaries.

LLM-guided strategies underperform in multi-document settings. Contrary to expectations, sophisticated LLM-guided approaches (LumberChunker and proposition-level chunking) do not outperform simpler chunkers. Proposition-based chunking consistently yields the lowest average scores across all models (ranging from 0.3487 to 0.3916), while LumberChunker ranks fourth or fifth in average corpus effectiveness. For instance, with Jina-v3, paragraph chunking outperforms LumberChunker by 11% (0.4948 vs 0.4463) and proposition-level chunking by 26% (0.4948 vs 0.3916).

Dataset characteristics influence chunking effectiveness. Effectiveness patterns vary across different corpus types. For domain-specific datasets like SciFact and TREC-COVID, sentence-based chunking often performs competitively or best (e.g., SciFact with Jina-v2: 0.6548, TREC-COVID with Nomic: 0.7859 for fixed-size). In contrast, for argumentative text (ArguAna), paragraph and fixed-size chunking consistently outperform more sophisticated strategies, suggesting that natural discourse boundaries are crucial for certain retrieval tasks.

Model architecture affects chunking sensitivity. The variance in effectiveness across chunkers differs substantially between models. Jina-v2 shows the highest sensitivity to chunking choice with a 23% effectiveness gap between best and worst strategies (0.4303 vs 0.3487), while E5-Large shows more robustness with only a 11% gap (0.4997 vs 0.3883). This indicates that some embedding models are inherently better at handling suboptimal chunk boundaries.

Overall, our results reveal that simple chunking strategies are most effective for multi-document retrieval. Paragraph-based chunking emerges as the most reliable choice, achieving the highest average effectiveness in three of four models and consistently ranking in the top-2 across individual datasets. Fixed-size chunking offers comparable effectiveness (within 2% of paragraph chunking on average) with computational advantages. Surprisingly, LLM-guided strategies significantly underperform despite their sophistication: proposition-level chunking averages 15-20% lower effectiveness than paragraph chunking across all models. This answers RQ1 by demonstrating that for standard multi-document retrieval, simpler chunkers that respect natural document boundaries or use consistent sizes provide superior effectiveness and robustness compared to semantic-aware alternatives.

5 RQ2: Do Chunking Strategies Generalize Across Retrieval Paradigms?

We contrast effectiveness on the single-document GutenQA benchmark with the multi-document BEIR tasks to evaluate cross-paradigm robustness. Table 1 shows that the chunkers excelling on GutenQA are precisely those that falter on BEIR. Proposition-level chunking delivers the highest GutenQA scores for three of the four encoders (e.g., 0.5556 DCG@10 with Jina-v2 and 0.5530 with E5-Large), yet it yields the lowest average corpus effectiveness in every case, trailing paragraph chunking by 18% relative for Jina-v2 (0.3487 vs. 0.4303) and by 22% for E5-Large (0.3883 vs. 0.4997). LLM-guided LumberChunker exhibits a similar pattern: it leads the GutenQA column for Jina-v3 (0.5693) but drops to fourth place on BEIR (0.4463 average). In contrast, simple heuristics, namely paragraph and fixed-size chunking, rarely win on GutenQA but consistently occupy the top two ranks on the multi-document tasks.

These trends persist when we switch to contextualized encoders in Table 2. Late chunking boosts proposition-based scores on GutenQA (0.5280 with Jina-v2, 0.5117 with E5-Large), yet its BEIR averages remain far below those of the

Table 1: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (regular encoder)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	0.3893	**0.3347**	**0.3042**	0.6389	0.6518	0.4682	0.1843	**0.4303**
	Sentence	0.3261	0.3096	0.2847	**0.6548**	**0.6674**	0.4675	0.1732	0.4262
	Fixed-Size	0.3889	0.3339	0.2604	0.6350	0.6437	**0.4706**	**0.1853**	0.4215
	Semantic	0.3858	0.3030	0.2746	0.6445	0.6618	0.4022	0.1760	0.4104
	Lumber	0.4782	0.2754	0.2887	0.6540	0.6456	0.3906	0.1721	0.4044
	Proposition	**0.5556**	0.2186	0.2995	0.6331	0.4292	0.3681	0.1436	0.3487
	Avg	0.4207	0.2959	0.2854	0.6434	0.6166	0.4279	0.1724	0.4069
Jina-v3	Paragraph	0.4574	**0.4747**	**0.3663**	0.7247	**0.7721**	**0.4327**	0.1984	**0.4948**
	Sentence	0.3877	0.4333	0.3566	0.7163	0.7135	0.4256	0.1888	0.4723
	Fixed-Size	0.4827	0.4728	0.3596	**0.7262**	0.7557	0.4325	**0.1985**	0.4909
	Semantic	0.4105	0.4396	0.3596	0.7103	0.7474	0.3894	0.1890	0.4726
	Lumber	**0.5693**	0.4028	0.3534	0.6845	0.6900	0.3671	0.1797	0.4463
	Proposition	0.5484	0.2858	0.3579	0.6690	0.5756	0.3044	0.1569	0.3916
	Avg	0.4760	0.4182	0.3589	0.7052	0.7091	0.3920	0.1852	0.4614
Nomic	Paragraph	0.4893	0.3802	0.3509	0.7047	0.7652	0.4904	0.1808	0.4787
	Sentence	0.4146	0.3620	0.3502	**0.7150**	0.7472	**0.5084**	0.1732	0.4760
	Fixed-Size	0.5104	**0.3839**	0.3492	0.7050	**0.7859**	0.4915	**0.1812**	**0.4828**
	Semantic	0.4274	0.3562	0.3506	0.7040	0.7431	0.4434	0.1707	0.4613
	Lumber	**0.6048**	0.3347	**0.3513**	0.7093	0.7193	0.4315	0.1673	0.4522
	Proposition	0.5827	0.2575	0.3358	0.6853	0.5580	0.3693	0.1441	0.3916
	Avg	0.5049	0.3457	0.3480	0.7039	0.7198	0.4557	0.1696	0.4571
E5-Large	Paragraph	0.4663	**0.4617**	0.3566	**0.7129**	0.6634	**0.5999**	0.2038	**0.4997**
	Sentence	0.4063	0.4315	0.3583	0.6873	0.6712	0.5646	0.1836	0.4828
	Fixed-Size	0.4862	0.4599	0.3495	0.7078	0.6590	0.5987	**0.2058**	0.4968
	Semantic	0.3186	0.4173	0.3583	0.6892	0.6784	0.5540	0.1888	0.4810
	Lumber	0.5234	0.3739	0.3531	0.6748	**0.6931**	0.5233	0.1792	0.4663
	Proposition	**0.5530**	0.2522	**0.3602**	0.6851	0.5611	0.3163	0.1551	0.3883
	Avg	0.4590	0.3994	0.3560	0.6928	0.6544	0.5261	0.1861	0.4691

heuristic baselines (0.4049 and 0.4030, respectively). Sentence blocks and fixed-size windows, while modest on document retrieval, retain their edge on BEIR after contextualization (e.g., 0.4400 for Jina-v2 sentence blocks, 0.4770 for Nomic sentence blocks). The lack of a single method that performs strongly in both paradigms demonstrates that chunkers tuned for precise passage localization do not generalize to open-domain retrieval, whereas coarse structural heuristics are more transferable even if they sacrifice some in-document fidelity.

Table 2: Effectiveness comparison across different models, chunking strategies, and datasets using DCG@10 for GutenQA and nDCG@10 for other datasets (late encoder)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	0.3476	0.3346	0.3041	0.6397	0.6618	0.4666	0.1854	0.4320
	Sentence	0.3084	**0.3392**	0.3033	0.6583	**0.6791**	0.4764	0.1839	**0.4400**
	Fixed-Size	0.3799	0.3351	0.3003	0.6448	0.6649	0.4685	**0.1858**	0.4332
	Semantic	0.4041	0.2195	0.2900	0.6489	0.6076	0.4715	0.1836	0.4035
	Lumber	0.4290	0.2223	0.3079	0.6527	0.6516	0.4759	0.1855	0.4160
	Proposition	**0.5280**	0.3042	**0.3210**	**0.6632**	0.4884	**0.4869**	0.1655	0.4049
	Avg	0.3995	0.2925	0.3044	0.6513	0.6256	0.4743	0.1816	0.4216
Jina-v3	Paragraph	0.2816	0.4306	0.3442	0.6826	0.6921	**0.4732**	0.2034	0.4710
	Sentence	0.2524	**0.4355**	0.3443	0.6841	**0.7046**	0.4605	0.2011	**0.4717**
	Fixed-Size	0.3411	0.4308	0.3442	0.6856	0.6881	0.4732	**0.2036**	0.4709
	Semantic	0.3356	0.2895	0.3474	0.6880	0.6527	0.4729	0.2019	0.4421
	Lumber	0.3789	0.2857	0.3506	**0.6947**	0.6826	0.4691	0.2020	0.4474
	Proposition	**0.4984**	0.3670	**0.3551**	0.6937	0.5229	0.4328	0.1883	0.4266
	Avg	0.3480	0.3732	0.3476	0.6881	0.6571	0.4636	0.2001	0.4550
Nomic	Paragraph	0.1839	**0.3801**	0.3508	0.7057	0.7361	0.4936	**0.1808**	0.4745
	Sentence	0.1557	0.3777	0.3536	0.7129	**0.7517**	0.4883	0.1778	**0.4770**
	Fixed-Size	0.2127	0.3796	0.3515	0.7058	0.7316	**0.4941**	0.1806	0.4738
	Semantic	0.2435	0.2804	0.3535	0.7036	0.7052	0.4890	0.1786	0.4517
	Lumber	0.2395	0.2757	**0.3604**	**0.7191**	0.6790	0.4823	0.1796	0.4494
	Proposition	**0.4213**	0.3145	0.3515	0.7124	0.4502	0.4773	0.1530	0.4098
	Avg	0.2428	0.3347	0.3535	0.7099	0.6756	0.4874	0.1751	0.4560
E5-Large	Paragraph	0.4223	**0.4613**	**0.3576**	**0.7152**	**0.6630**	**0.5994**	**0.2064**	**0.5005**
	Sentence	0.3621	0.4608	0.3553	0.7030	0.6479	0.5945	0.2061	0.4946
	Fixed-Size	0.4863	0.4601	0.3510	0.7095	0.6615	0.5978	0.2063	0.4977
	Semantic	0.3259	0.3227	0.3549	0.7006	0.6430	0.5923	0.2055	0.4698
	Lumber	**0.5245**	0.3015	0.3518	0.6976	0.6137	0.5922	0.2050	0.4603
	Proposition	0.5117	0.3306	0.3405	0.6427	0.4160	0.5056	0.1826	0.4030
	Avg	0.4388	0.3895	0.3518	0.6948	0.6075	0.5803	0.2020	0.4710

6 RQ3: Effectiveness of Contextualized Chunking

We next examine whether contextualized chunking (late encoder) provides consistent gains over pre-embedding baselines. Table 3 reports relative changes when switching from the regular encoder to the late encoder. Improvements are highly localized: sentence blocks with Jina-v2 gain 6.5% on nfcorpus and 9.6% on FiQA, while losing 5.4% DCG@10 on GutenQA. Fixed-size windows also benefit slightly on nfcorpus (+15.3%) yet deteriorate on the document task (-2.3%). Paragraph chunking, which is the strongest BEIR baseline, rarely improves; its average change across corpora is within +/-0.5% for three encoders, and it suffers double-digit drops on GutenQA (e.g., -10.7% for Jina-v2, -39.4% for Nomic).

LLM-driven methods experience the largest swings. Proposition-level chunking enjoys dramatic relative gains on argumentative corpora when paired with contextualized embeddings (e.g., +42.2% nDCG on ArguAna with Jina-v2 and +59.8% with E5-Large), but those gains coincide with severe regressions on cleaner domains (-25.9% on TREC-COVID for E5-Large, -19.3% for Nomic). LumberChunker similarly oscillates: contextualization boosts ArguAna by up to +27.8% for Jina-v2 yet erodes FiQA by as much as -29.1%.

Overall, the late encoder delivers at most marginal average gains across BEIR (no more than +2.4 percentage points) while incurring substantial cost on the single-document task (up to -62.5% for paragraph chunking with Nomic). These results indicate that contextualized chunking is not a free upgrade: it must be targeted at specific chunker-model-dataset combinations where the additional context outweighs the noise introduced by pooling across long sequences.

7 RQ4: Other Factors

Acknowledgment

References

Table 3: Late-encoder effectiveness relative to the regular encoder (baseline). Values show percentage change (light green=improvement, light red=degradation)

		document	***corpus***						
		GutenQA	fiqa	nfcorpus	scifact	trec-covid	arguana	scidocs	Avg
Jina-v2	Paragraph	-10.72	-0.02	-0.03	+0.12	+1.53	-0.33	+0.61	+0.31
	Sentence	-5.43	+9.57	+6.52	+0.52	+1.76	+1.89	+6.17	+4.40
	Fixed-Size	-2.32	+0.35	**+15.32**	+1.55	+3.30	-0.44	+0.27	+3.39
	Semantic	**+4.75**	-27.57	+5.59	+0.69	-8.19	+17.22	+4.31	-1.32
	Lumber	-10.30	-19.29	+6.66	-0.19	+0.93	+21.85	+7.80	+2.96
	Proposition	-4.98	**+39.17**	+7.18	**+4.75**	**+13.79**	**+32.26**	**+15.29**	**+18.74**
	Avg	-4.83	+0.37	+6.87	+1.24	+2.19	+12.07	+5.74	+4.75
Jina-v3	Paragraph	-38.43	-9.28	-6.02	-5.81	-10.37	+9.37	+2.52	-3.27
	Sentence	-34.89	+0.51	-3.43	-4.49	-1.25	+8.20	+6.55	+1.01
	Fixed-Size	-29.35	-8.88	-4.27	-5.58	-8.95	+9.40	+2.57	-2.62
	Semantic	-18.24	-34.15	-3.41	-3.14	-12.67	+21.43	+6.85	-4.18
	Lumber	-33.46	-29.08	-0.78	+1.48	-1.08	+27.78	+12.40	+1.79
	Proposition	-9.12	**+28.41**	-0.78	**+3.70**	-9.16	**+42.20**	**+20.01**	**+14.06**
	Avg	-27.25	-8.75	-3.11	-2.31	-7.25	+19.73	+8.48	+1.13
Nomic	Paragraph	-62.41	-0.03	-0.03	+0.14	-3.80	+0.65	+0.02	-0.51
	Sentence	-62.46	+4.33	+0.98	-0.28	**+0.60**	-3.96	+2.66	+0.72
	Fixed-Size	-58.33	-1.10	+0.63	+0.11	-6.91	+0.51	-0.36	-1.19
	Semantic	-43.04	-21.28	+0.82	-0.04	-5.09	+10.29	+4.62	-1.78
	Lumber	-60.40	-17.64	+2.61	+1.39	-5.59	+11.78	**+7.35**	-0.02
	Proposition	-27.70	**+22.15**	**+4.68**	**+3.95**	-19.32	**+29.26**	+6.22	**+7.82**
	Avg	-52.39	-2.26	+1.62	+0.88	-6.69	+8.09	+3.42	+0.84
E5-Large	Paragraph	-9.43	-0.09	+0.27	+0.32	-0.05	-0.08	+1.27	+0.27
	Sentence	-10.89	+6.80	-0.84	+2.29	-3.48	+5.29	+12.28	+3.72
	Fixed-Size	+0.01	+0.05	**+0.42**	+0.25	**+0.38**	-0.14	+0.25	+0.20
	Semantic	**+2.29**	-22.66	-0.96	+1.66	-5.22	+6.92	+8.83	-1.90
	Lumber	+0.20	-19.37	-0.39	**+3.37**	-11.46	+13.18	+14.38	-0.05
	Proposition	-7.47	**+31.09**	-5.47	-6.18	-25.85	**+59.84**	**+17.77**	**+11.87**
	Avg	-4.22	-0.69	-1.16	+0.28	-7.61	+14.17	+9.13	+2.35